

```
#include "system.h"
#include "io.h"
#include "alt_types.h"

#define RTC_ADR 0x68
#define byte unsigned char

#define START 1 << 10
#define REG 1 << 9
#define STOP 1 << 8
#define READ 0x1

// DEFINE DS 3231 REGISTERS
#define SECONDS 0x0
#define MINUTES 0x1
#define HOURS 0x2
#define WEEKDAY 0x3
#define DATE 0x4
#define MONTH 0x5
#define YEAR 0x6
#define TEMP_HIGH 0x11
#define TEMP_LOW 0x12

// define I2C IP CORE REGISTERS
#define CTRL 0*2
#define STATUS 1*2
#define RD_Data 2*2
#define WR_Data 3*2

// SCL SPEEED, 100khz/400khz
#define I2C_MODE_STANDARD 0
#define I2C_MODE_FAST 1
// Default scl speed and slave address
int i2c_mode = I2C_MODE_STANDARD;
byte SLAVE_ADDRESS = RTC_ADR;

void I2C_WAIT_BUSY() {
    while ((IORD_16DIRECT(I2C_MASTER_0_BASE, STATUS) & 1) == 1);
} // wait until FSM is back in idle state

void i2c_set_mode(int mode) {
    i2c_mode = mode;
} // set SCL mode (0 or 1)

void i2c_set_slave_adr(byte slave_adr) {
    SLAVE_ADDRESS = slave_adr;
} // Change slave adress if you want to use something other than ds3231

void i2c_ctrl(int threebyte) {
    IOWR_16DIRECT(I2C_MASTER_0_BASE, CTRL, (i2c_mode << 1) | threebyte);
} // set CTRL register, input threebyte = 1, or 0 for two bytes

byte dectobcd(byte dec) {
```

```
    byte tiere = dec / 10;
    byte enere = dec % 10;
    return (tiere * 16) + enere;
} // converts numbers from 0-99 to bcd

void i2c_write2(byte register_address) {
    i2c_ctrl(0);
    I2C_WAIT_BUSY();
    IOWR_16DIRECT(I2C_MASTER_0_BASE, WR_Data, START | (SLAVE_ADDRESS << 1));
    IOWR_16DIRECT(I2C_MASTER_0_BASE, WR_Data, REG | register_address);
} // write 2 bytes

void i2c_write3(byte register_address, byte data) {
    i2c_ctrl(1);
    I2C_WAIT_BUSY();
    IOWR_16DIRECT(I2C_MASTER_0_BASE, WR_Data, START | (SLAVE_ADDRESS << 1));
    IOWR_16DIRECT(I2C_MASTER_0_BASE, WR_Data, REG | register_address);
    IOWR_16DIRECT(I2C_MASTER_0_BASE, WR_Data, STOP | data);
} // write 3 bytes

byte i2c_read_register(byte REGISTER_ADDRESS) {
    i2c_ctrl(0);
    I2C_WAIT_BUSY();
    IOWR_16DIRECT(I2C_MASTER_0_BASE, WR_Data, START | (SLAVE_ADDRESS << 1));
    IOWR_16DIRECT(I2C_MASTER_0_BASE, WR_Data, STOP | REGISTER_ADDRESS);

    I2C_WAIT_BUSY();
    IOWR_16DIRECT(I2C_MASTER_0_BASE, WR_Data, START | (SLAVE_ADDRESS << 1) | READ);
    IOWR_16DIRECT(I2C_MASTER_0_BASE, WR_Data, STOP);

    I2C_WAIT_BUSY();
    return IORD_16DIRECT(I2C_MASTER_0_BASE, RD_Data) & 0xFF;
} // read i2c register

void set_rtc_time(byte second, byte minute, byte hour, byte week_day, byte day,
byte month, byte year) {
    i2c_ctrl(1);
    i2c_set_slave_adr(RTC_ADR);
    byte datatid[] = {second, minute, hour, week_day, day, month, year};
    for (int i = 0; i < 7; i++) {
        i2c_write3(SECONDS + i, dectobcd(datatid[i]));
    } // set rtc time in ds3231
}

void get_rtc_time(byte *second, byte *minute, byte *hour, byte *week_day, byte *
*day, byte *month, byte *year) {
    i2c_ctrl(1);
    i2c_set_slave_adr(RTC_ADR);
    byte *pData[] = {second, minute, hour, week_day, day, month, year};
    for (int i = 0; i < 7; i++) {
        byte raw_bcd = i2c_read_register(SECONDS + i);
        byte tiere = raw_bcd >> 4;
        byte enere = raw_bcd & 0x0F;
```

```
        *pData[i] = tiere * 10 + enere;
    } // get rtc time in ds3231
}

float get_rtc_temp(void) {
    i2c_ctrl(1);
    i2c_set_slave_adr(RTC_ADR);
    byte temp[2];
    for (int i = 0; i < 2; i++) {
        temp[i] = i2c_read_register(TEMP_HIGH + i);
    }
    short int temp_int = (temp[0] << 2) | (temp[1] >> 6);
    return temp_int / 4.0;
} // get rtc temperature in ds3231
```